

PONTIFÍCIA UNIVERSIDADE CATÓLICA
CIÊNCIA DA COMPUTAÇÃO

ANE M. VIANA, CAMILA C. MENEZES, GUILHERME A. ZUIM, MARIA CLARA
G. SOARES

HASKELL - LINGUAGEM DE PROGRAMAÇÃO

BELO HORIZONTE, MINAS GERAIS
2026

Sumário

1	INTRODUÇÃO	2
2	HISTÓRICO	2
3	PARADIGMAS	3
4	CARACTERÍSTICAS MARCANTES	5
5	LINGUAGENS RELACIONADAS	5
6	EXEMPLOS DE PROGRAMAS	6
6.1	Dobro de um número	6
6.2	Verificação de número par	6
6.3	Soma de elementos de uma lista	6
6.4	Filtro de nomes com mais de 5 letras	7
6.5	Sequência de Fibonacci (Lazy Evaluation)	7
6.6	Simulação de consulta em banco de dados	7
7	PRÁTICA	7
7.1	Fatorial	7
7.2	QuickSort	8
7.3	Composição de Funções	8
7.4	Função Currificada	8
8	CONSIDERAÇÕES FINAIS	8
9	APÊNDICE	9
9.1	Ane Madjarian	9
9.2	Camila Menezes	9
9.3	Maria Clara Galvão	10
9.4	Guilherme Zuim	11

1 INTRODUÇÃO

O desenvolvimento da computação moderna é frequentemente narrado sob a ótica da evolução do hardware e das linguagens imperativas, que mimetizam o funcionamento da arquitetura de Von Neumann através de estados mutáveis e sequenciamento de comandos. No entanto, paralelamente a essa trajetória, consolidou-se um pilar matemático fundamental que precede os próprios computadores eletrônicos: o Cálculo Lambda (λ -calculus), formulado por Alonzo Church na década de 1930. É neste cenário de rigor matemático e busca por abstração que se insere a linguagem Haskell, uma linguagem puramente funcional, de tipagem estática e avaliação preguiçosa, que representa o ápice da aplicação prática de conceitos teóricos da lógica combinatória e da teoria das categorias na programação. Diferente do paradigma imperativo, onde o foco reside em como o computador deve alterar seu estado interno, o Haskell prioriza a declaração de o que o problema representa. Através da pureza funcional, a linguagem assegura que as funções não possuam efeitos colaterais, garantindo que, para uma mesma entrada, o resultado seja invariavelmente o mesmo. Essa previsibilidade é o que torna o Haskell uma ferramenta excepcional para a criação de sistemas de alta confiabilidade, onde a correção lógica é verificada rigorosamente pelo sistema de tipos Hindley-Milner durante a compilação. Além disso, a característica de avaliação não-estrita (lazy evaluation) permite que o desenvolvedor trabalhe com estruturas de dados potencialmente infinitas, separando o controle de fluxo da geração de dados de maneira modular e eficiente. Atualmente, o Haskell transcende o ambiente acadêmico, influenciando linguagens modernas e sustentando infraestruturas críticas em setores financeiros e de segurança cibernética, onde a imutabilidade e o paralelismo natural são requisitos fundamentais. Este trabalho explora tais pilares técnicos, detalhando as características que definem a linguagem e analisando o artigo [A History of Haskell: Being Lazy with Class], que demonstra a aplicação prática desses conceitos em desafios contemporâneos da computação.

2 HISTÓRICO

A linguagem Haskell tem sua origem em um movimento acadêmico que buscava consolidar e padronizar as diversas linguagens funcionais que surgiram ao longo das décadas de 1970 e 1980. Nesse período, diferentes propostas — como ML e Miranda — exploravam conceitos avançados de tipagem e avaliação, mas ainda careciam de uma unificação que permitisse maior comparabilidade entre pesquisas e aplicações.

Em 1987, durante a conferência Functional Programming Languages and Computer Architecture, pesquisadores da área reconheceram a fragmentação existente e propuseram a criação de uma linguagem comum, aberta e baseada em fundamentos teóricos sólidos. A iniciativa reuniu especialistas de diferentes instituições com o objetivo de projetar uma linguagem que servisse tanto para pesquisa quanto para ensino, incorporando o que havia de mais avançado no paradigma funcional.

O nome Haskell foi escolhido em homenagem a Haskell Brooks Curry, cujas contribuições à lógica matemática influenciaram profundamente o desenvolvimento da programação funcional. A primeira especificação oficial, conhecida como Haskell 1.0, foi publicada em 1990, marcando o início de um processo contínuo de refinamento da linguagem. Ao longo da década de 1990, novas versões foram lançadas, ajustando aspectos da sintaxe, do sistema de tipos e das bibliotecas padrão.

Esse processo culminou na definição do padrão Haskell 98, que estabeleceu uma

base estável e amplamente adotada. Paralelamente, surgiram implementações robustas, com destaque para o Glasgow Haskell Compiler, que se tornou a principal ferramenta de desenvolvimento e desempenhou papel central na evolução prática da linguagem.

Na década seguinte, o esforço de atualização levou à especificação Haskell 2010, que incorporou extensões já consolidadas na prática, mantendo a coerência com os princípios originais da linguagem. Desde então, o desenvolvimento do Haskell tem ocorrido de forma incremental, principalmente por meio de extensões implementadas em compiladores e bibliotecas, permitindo sua adaptação a novos contextos sem comprometer sua base teórica. Na seção 2 terá os materiais e métodos utilizados; na seção 3 a fundamentação teórica; na seção 4 será apresentado os resultados obtidos e discuti-los; na seção 5 terá a conclusão.

3 PARADIGMAS

A linguagem Haskell é uma linguagem puramente funcional, ou seja, seu funcionamento é completamente contrário a linguagens imperativas. Miran Lipovača (2011) define que em linguagens imperativas uma pessoa faz uma declaração clara do que o programa deve fazer para resolver algum problema, enquanto em linguagens puramente funcionais é dito ao computador o que algo é. Desta forma, a linguagem Haskell é uma linguagem que segue a lógica de definição de funções, sem de fato apresentar a ela a sequência de fatos que ela deve estritamente seguir para chegar a uma solução, funcionando em uma sequência de funções interligadas entre si, e nas quais, não podem mudar o valor de suas variáveis quando já definidas em um instante do programa.

Estas funções também possuem o que é chamado de transparência referencial, ou seja, seus parâmetros sendo os mesmos valores, não ocorre nenhuma mudança ao chamar a função em mais um momento, o resultado será o mesmo, e a preocupação da função está em calcular e retornar um valor, e de acordo com Miran Lipovača (2011), em linguagens funcionais não há variáveis mutáveis, uma vez definida, não é possível mudá-la, e as funções criadas não possuem efeitos colaterais, somente podem calcular e retornar valores, trabalhando com transparência referencial.

Estas e outras características da linguagem Haskell foram muito importantes para a sua construção e definição de paradigma, que a define como é, e serão apresentadas melhor ao longo desta seção.

Uma das principais características deste paradigma e da linguagem Haskell, é a imutabilidade. Em Haskell, por ser uma linguagem pura, as variáveis não são mutáveis, uma vez que se é definido seu valor, não é possível reatribuir um novo valor a ela, assim como em uma função matemática dentro do programa, ela sempre retornará o mesmo valor que lhe foi atribuído. Embora pareça ser limitada, esta característica possui pontos positivos, pois, permite que você chame uma mesma função inúmeras vezes, sem que o resultado seja mudado, e construir funções mais complexas a partir disso.

Outro fator importante, é que a linguagem Haskell é “preguiçosa”, ou seja, a linguagem somente irá realizar alguma função se for necessário o seu resultado, e isso é possível devido a transparência referencial. Segundo Hudak et al. (2007), Haskell possui uma semântica não-estrita, o que no uso popular é a “preguiça”, e isto seria dizer que a linguagem não é uma linguagem que atua de forma imediata ao avaliar expressões de forma estrita.

Entretanto, isto é pouco eficiente em relação a chamada por valor, e gera mais trabalho para atrasar avaliações e sobrescrever termos. Porém, um problema muito maior,

segundo Hudak et al. (2007), é a imprevisibilidade do programador sobre o comportamento de espaço da avaliação, o que frequentemente resulta no predomínio de vazamentos de espaço de memória (space leaks), mas, para lidar com essa questão, os autores explicam que foi necessário adicionar alguns recursos estritos à linguagem, como a função seq e tipos de dados estritos, provando que a divisão entre uma linguagem puramente estrita ou preguiçosa não precisa ser uma decisão de "tudo ou nada".

Além disto, Haskell é uma linguagem estaticamente tipada, ou seja, os erros e os tipos das variáveis são reconhecidos e diferenciados em tempo de compilação. Ela também possui inferência de tipo, isto é, a linguagem consegue descobrir os tipos de forma automática, não sendo necessário especificá-los manualmente, o que facilita a criação dos códigos. Toda essa base de tipagem estática e forte, junto à inferência, foi inspirada na linguagem Miranda, que utiliza o sistema Hindley-Milner. É esse sistema que permite que o código seja escrito de forma genérica para lidar com múltiplos tipos de dados com total segurança.

De acordo com Miran Lipovača (2011), a linguagem Haskell possui funções conhecidas como funções de alta ordem, isto é, funções que possuem funções como parâmetros, e que retornam funções como se fossem valores, e são bastante úteis na resolução de problemas em programação funcional em Haskell. Em complemento, as funções em Haskell são nativamente currificadas, isto é, cada função aceita apenas um parâmetro por vez, e quando múltiplos argumentos são fornecidos, a função processa o primeiro e retorna uma nova função que aguarda o próximo, e assim sucessivamente. Esse mecanismo permite a aplicação parcial, isto é, criar uma função nova apenas “travando” alguns parâmetros de uma função que já existe. Isso facilita a composição de funções, onde foca-se em definir o que o código é em vez de escrever um passo a passo de como processar os dados, tornando assim o código limpo, modular e sem uso de variáveis temporárias.

Diante das características apresentadas, é possível identificar as vantagens do uso do paradigma funcional adotado pela linguagem Haskell. Para Miran Lipovača (2011), Haskell é uma linguagem concisa e elegante, que permite criar códigos menores do que em linguagens imperativas, trazendo conceitos de alto nível. Assim, ela se mostra altamente confiável graças à sua verificação em tempo de compilação, capturando a maior parte dos erros antes mesmo da execução. Além disso, é uma ferramenta muito produtiva, pois permite que o programador se concentre puramente em resolver os problemas propostos, sem a necessidade de focar na escrita de códigos repetitivos.

Por outro lado, apesar dos benefícios, esse paradigma também apresenta alguns desafios. Ela é uma linguagem mais difícil de aprender por pessoas que estão acostumadas com linguagens imperativas como Java, por exemplo. Além disso, por ser uma linguagem que avalia por demanda, o processo de depuração do código torna-se algo difícil de aplicar na prática. Como a ordem de execução não é facilmente previsível, ferramentas convencionais de rastreamento geram resultados confusos. Essa mesma característica de avaliação preguiçosa cria enormes desafios no controle de memória, favorecendo o surgimento de vazamentos de espaço difíceis de prever. Para completar a curva de aprendizado, iniciantes frequentemente enfrentam um período de dificuldade até entenderem quais tipos de bugs geram certas mensagens de erro incompreensíveis do compilador.

4 CARACTERÍSTICAS MARCANTES

A principal característica marcante da linguagem de programação Haskell é o fato de ela ser uma linguagem puramente funcional. Em contraste com linguagens imperativas estudadas ao longo dos últimos semestres, como C ou Java, Haskell é baseada no uso de funções matemáticas puras, sem estados mutáveis. Isso significa que, ao atribuir um valor a uma variável, esse valor permanece constante e não é alterado ao longo da execução do programa. Essa característica permite o conceito de transparência referencial, no qual uma função chamada duas vezes com o mesmo parâmetro sempre retorna o mesmo resultado.

Outra característica importante de Haskell é sua tipagem estática. Isso significa que os tipos das variáveis e expressões são verificados durante a compilação, permitindo que diversos erros sejam detectados antes da execução do programa. Além disso, Haskell possui inferência de tipos, o que significa que o programador não precisa declarar explicitamente o tipo de todas as variáveis. O compilador é capaz de deduzir automaticamente o tipo de uma expressão a partir do contexto. Por exemplo, na expressão $a = 2 + 1$, mesmo sem especificar o tipo de a , o compilador consegue identificar que se trata de um valor numérico. A inferência de tipos também possibilita que funções sejam mais genéricas, funcionando para diferentes tipos que possuam comportamentos semelhantes.

Haskell também é considerada uma lazy language, ou seja, utiliza o conceito de lazy evaluation (avaliação preguiçosa). Nesse modelo de execução, expressões e funções não são avaliadas imediatamente, mas apenas quando seu resultado é realmente necessário, como no momento em que o programa precisa produzir uma saída. Inicialmente, o programa armazena uma espécie de promessa de cálculo, conhecida como thunk, que só será avaliada quando seu valor for requerido. Comparando com linguagens como C, onde as funções são avaliadas imediatamente no momento da chamada, a avaliação preguiçosa evita cálculos desnecessários e permite otimizações no processamento de dados. Esse mecanismo também possibilita a criação de estruturas de dados potencialmente infinitas, como listas infinitas, que são avaliadas apenas parcialmente conforme necessário.

Por fim, programas escritos em Haskell costumam ser considerados conciso e elegantes. Devido ao uso de conceitos de alto nível e à natureza declarativa da linguagem, os programas geralmente são menores do que implementações equivalentes em linguagens imperativas. Essa concisão contribui para uma maior legibilidade do código, além de facilitar sua manutenção e reduzir a probabilidade de erros.

5 LINGUAGENS RELACIONADAS

O desenvolvimento da Haskell não ocorreu de forma isolada, mas sim como parte de uma tradição contínua dentro da programação funcional. Diversas linguagens influenciaram sua concepção, ao mesmo tempo em que outras foram posteriormente impactadas por suas ideias e inovações.

Entre as principais influências históricas, destaca-se a ML, que introduziu um sistema de tipos com inferência automática, elemento fundamental que viria a ser incorporado e refinado em Haskell. Outra linguagem relevante é a Miranda, que antecipou características como a avaliação preguiçosa e a sintaxe baseada em funções puras, servindo como uma base direta para várias decisões de design adotadas posteriormente.

No que se refere às linguagens contemporâneas, Haskell exerceu forte influência sobre diversas tecnologias modernas. A Scala, por exemplo, combina paradigmas funcio-

nal e orientado a objetos, incorporando conceitos como funções de alta ordem e tipagem avançada. De forma semelhante, a F adapta ideias da programação funcional ao ecossistema .NET, facilitando sua adoção em ambientes corporativos.

Outras linguagens mais recentes, como Rust e Kotlin, embora não sejam puramente funcionais, absorveram princípios importantes difundidos por Haskell, como imutabilidade, segurança de tipos e maior expressividade na definição de funções. Esses elementos demonstram como conceitos oriundos da programação funcional passaram a influenciar o desenvolvimento de software de forma mais ampla.

Além disso, linguagens funcionalmente puras ou fortemente inspiradas em Haskell, como Elm, evidenciam a continuidade dessa linha de pensamento, especialmente em contextos como desenvolvimento web, onde previsibilidade e ausência de efeitos colaterais são altamente valorizadas.

6 EXEMPLOS DE PROGRAMAS

6.1 Dobro de um número

```
dobro :: Int -> Int
dobro x = x * 2
```

A função é definida de forma declarativa, indicando apenas a relação entre entrada e saída. Haskell não exige descrição passo a passo do processo.

Em C, seria necessário declarar explicitamente o tipo da função e utilizar uma estrutura imperativa com retorno, tornando o código mais detalhado.

6.2 Verificação de número par

```
ehPar :: Int -> Bool
ehPar n = mod n 2 == 0
```

A função é uma expressão pura, sem efeitos colaterais, garantindo que a mesma entrada sempre produzirá a mesma saída.

Em C, apesar da lógica ser semelhante, a linguagem permite alterações de estado com facilidade, o que pode reduzir a previsibilidade do código.

6.3 Soma de elementos de uma lista

```
somaLista :: [Int] -> Int
somaLista [] = 0
somaLista (x:xs) = x + somaLista xs
```

Haskell utiliza recursão e *pattern matching* para percorrer listas de forma natural, sem necessidade de controle de índices.

Em C, essa operação geralmente é realizada com arrays e estruturas de repetição, exigindo controle manual de posições e tamanho.

6.4 Filtro de nomes com mais de 5 letras

```
filtrarNomes :: [String] -> [String]
filtrarNomes nomes = filter (\n -> length n > 5) nomes
```

O uso da função `filter` evidencia a capacidade de Haskell de trabalhar com funções de ordem superior, permitindo maior abstração e concisão.

Em C, não há suporte nativo a esse tipo de abordagem, sendo necessário implementar a lógica manualmente com loops e condicionais.

6.5 Sequência de Fibonacci (Lazy Evaluation)

```
fibs :: [Int]
fibs = 0 : 1 : zipWith (+) fibs (drop 1 fibs)
```

A definição de uma lista infinita é possível devido à avaliação preguiçosa de Haskell, onde os valores são calculados apenas quando necessários.

Em C, não é possível definir estruturas infinitas dessa forma, sendo necessário limitar explicitamente a execução.

6.6 Simulação de consulta em banco de dados

```
usuarios :: [(Int, String)]
usuarios = [(1, "Guilherme"), (2, "Ane"), (3, "Maria"), (4, "Camila")]

buscarId :: Int -> [(Int, String)]
buscarId idLista = filter (\(i, nome) -> i == idLista) usuarios
```

A função simula uma consulta semelhante a um *SELECT*, utilizando filtragem declarativa sobre uma coleção de dados.

Em C, essa operação exigiria percorrer manualmente a estrutura com loops e comparações explícitas, reforçando o caráter imperativo da linguagem.

7 PRÁTICA

A seguir, são explicados os programas que foram executados na apresentação em sala de aula. Foi feito um menu interativo contendo os 4 programas, para que seja possível executar em tempo real e de forma dinâmica.

7.1 Fatorial

- **O que faz:** Calcula o produto de todos os números inteiros positivos menores ou iguais a n .
- **Como faz:** Utiliza casamento de padrões (*pattern matching*) para definir o caso base (`fatorial 0 = 1`). Para qualquer outro valor, a função chama a si mesma recursivamente (`n * fatorial (n - 1)`), substituindo o uso de estruturas iterativas por uma definição matemática.

7.2 QuickSort

- **O que faz:** Ordena uma lista de inteiros de forma crescente.
- **Como faz:** Utiliza a técnica de “dividir para conquistar”. Por meio de *list comprehensions*, a lista é dividida em duas sublistas: uma com elementos menores ou iguais ao pivô e outra com elementos maiores. Os resultados das chamadas recursivas são então combinados com o operador de concatenação (`++`), evidenciando a concisão do Haskell em comparação com linguagens imperativas.

7.3 Composição de Funções

- **O que faz:** Executa uma sequência de transformações em um dado (neste caso, soma 5 e depois dobra o valor).
- **Como faz:** Utiliza o operador de composição (`.`), permitindo combinar funções sem a necessidade de passar argumentos explicitamente. A função `dobroMaisCinco` é criada a partir da combinação das funções `dobro` e `somarCinco`, demonstrando como novas funções podem ser formadas a partir de funções existentes.

7.4 Função Currificada

- **O que faz:** Demonstra a criação de uma função especializada a partir de uma função genérica.
- **Como faz:** Em Haskell, todas as funções são currificadas por padrão. A função `soma` recebe dois argumentos, mas ao definir `somar10 = soma 10`, ocorre uma aplicação parcial. Assim, cria-se uma nova função que já possui um argumento fixado e espera apenas o próximo para completar o cálculo.

8 CONSIDERAÇÕES FINAIS

Ao longo deste trabalho, foi possível compreender a importância da linguagem Haskell no contexto da programação funcional, destacando-se por sua forte base teórica fundamentada no cálculo lambda e por suas características que a diferenciam das linguagens imperativas tradicionais. A proposta declarativa da linguagem, focada em definições de funções matemáticas puras, mostra uma forma diferente de se construir programas, priorizando a solução de problemas a sequência de instruções a serem resolvidas no algoritmo.

Além disso, foram analisadas as principais características que definem a linguagem Haskell como uma verdadeira linguagem puramente funcional, como a imutabilidade, a transparência referencial, a avaliação “preguiçosa”, e seu sistema de tipo forte e estático, que possui inferências. E tudo isto, contribui para a construção de códigos mais seguros, mais curtos, previsíveis e que possuem um alto nível de abstração e valoriza-se a reutilização de código. Por outro lado, a complexidade de aprendizado, a dificuldade de depuração e análise de código para correções de bugs e problemas, e a dificuldade em controle de memória pela avaliação não-estrita, é algo que dificulta bastante ao trabalhar com a linguagem.

Por fim, conclui-se que a linguagem Haskell possui grande importância para o avanço da aplicação prática de conceitos teóricos na computação, contribuindo significativamente para o desenvolvimento de pesquisas e sendo relevante em aplicações que exigem alta confiabilidade. Seu foco no uso de funções puras e na abstração matemática influenciou o surgimento e a evolução de outras linguagens, como Scala, Rust e Kotlin, além de ampliar a compreensão sobre algoritmos, programação e diferentes paradigmas existentes na computação.

9 APÊNDICE

9.1 Ane Madjarian

Na minha opinião, Haskell é uma linguagem que se destaca por propor uma forma completamente diferente de pensar a programação. Ao invés de focar em estados mutáveis e sequências de comandos, ela incentiva uma abordagem mais declarativa e matemática, o que pode ser desafiador no início, mas também extremamente enriquecedor.

O que eu mais gosto em Haskell é o fato de ela ser uma linguagem “preguiçosa” (lazy). Isso significa que as expressões só são avaliadas quando realmente necessário, o que abre espaço para soluções muito elegantes e eficientes. Com a avaliação preguiçosa, é possível, por exemplo, trabalhar com estruturas potencialmente infinitas, como listas infinitas, de forma natural e segura. Além disso, isso permite escrever códigos mais modulares, onde partes do programa só são executadas quando seus resultados são de fato utilizados.

Outro ponto interessante é que essa característica contribui para uma maior clareza no código. Como não há execução desnecessária, o programador tende a pensar melhor sobre dependências e fluxo de dados, resultando em soluções mais organizadas e, muitas vezes, mais fáceis de testar e manter.

Apesar de ter uma curva de aprendizado mais íngreme, especialmente para quem vem de linguagens imperativas, Haskell oferece uma experiência única. A combinação de programação funcional pura, tipagem forte e avaliação preguiçosa faz com que ela seja uma linguagem muito poderosa, principalmente para quem busca escrever código mais conciso, expressivo e correto.

9.2 Camila Menezes

Durante o desenvolvimento deste trabalho sobre a linguagem Haskell, um dos aspectos que mais chamou minha atenção foi a forma como o conteúdo da linguagem é apresentado no material *Learn You a Haskell for Great Good*. Diferente de abordagens tradicionais mais técnicas e formais, esse recurso utiliza uma didática criativa, visual e intuitiva, tornando o aprendizado mais leve e acessível, mesmo tratando de conceitos avançados da programação funcional.

A forma como os exemplos são apresentados, com ilustrações e explicações progressivas, contribui significativamente para a compreensão de conceitos que, à primeira vista, podem parecer abstratos, como funções puras, composição de funções e tipagem. Esse estilo de ensino evidencia que linguagens como Haskell, apesar de sua forte base matemática, podem ser aprendidas de maneira mais natural quando há uma boa mediação didática.

Outro ponto que despertou grande interesse foi o conceito de avaliação preguiçosa (*lazy evaluation*). Diferentemente das linguagens imperativas, como C, em que as expressões são avaliadas imediatamente no momento da execução, Haskell adia a avaliação até que o resultado seja realmente necessário. Esse comportamento permite evitar cálculos desnecessários e torna possível trabalhar com estruturas de dados potencialmente infinitas, como listas infinitas.

Esse modelo de execução representa uma mudança significativa na forma de pensar a programação. Em vez de focar na ordem de execução das instruções, o programador passa a se preocupar mais com a definição das relações entre os dados e as funções. Embora esse paradigma possa inicialmente causar estranhamento, especialmente para quem possui experiência prévia com linguagens imperativas, ele amplia a forma de enxergar a resolução de problemas computacionais.

Além disso, a *lazy evaluation* reforça a ideia de que Haskell prioriza eficiência conceitual e abstração, permitindo a construção de soluções elegantes e expressivas. No entanto, também evidencia desafios, como a dificuldade de prever o momento exato em que determinadas expressões serão avaliadas, o que pode impactar a compreensão do comportamento do programa.

Portanto, a combinação entre uma abordagem didática diferenciada, como a apresentada no *Learn You a Haskell for Great Good*, e conceitos inovadores como a avaliação preguiçosa, foi o que mais se destacou durante a realização deste trabalho. Esses elementos não apenas facilitaram o aprendizado da linguagem, mas também contribuíram para uma compreensão mais ampla sobre diferentes paradigmas de programação.

9.3 Maria Clara Galvão

O Haskell é uma linguagem que, num primeiro momento, ao realizar esta pesquisa, achei bem “estranha” e diferente, pois utiliza e possui características muito distintas em relação às linguagens vistas durante o primeiro e o segundo períodos do curso, como C, C++ e Java. O fato de não ser possível reatribuir valores a uma variável, ou mesmo guardar o estado de uma função, foi algo que me causou bastante estranheza, além da forma diferente de declarar as variáveis, embora esta última apresente pouca diferença estrutural em relação às linguagens imperativas que eu já conhecia.

Por outro lado, a forma como as funções são declaradas e utilizadas, o seu objetivo central e a própria estrutura dos programas foram coisas que me surpreenderam de forma muito positiva. Notei que as funções, em geral, são simples de declarar devido à facilidade na definição de parâmetros e retornos, utilizando apenas o sistema de setas e espaços. Além disso, o fato de ser possível utilizar funções como parâmetros e retorná-las como valores foi algo que achei muito interessante neste contexto de programação, permitindo uma dinâmica de código que eu ainda não tinha explorado.

Sobre a origem da linguagem, achei interessante o foco dos desenvolvedores em construir uma ferramenta voltada para o aprendizado e para a pesquisa no cenário da época em que foi criada. Esta abordagem pode facilitar muito a compreensão de conceitos sobre lógica e programação pura, sendo, em alguns aspectos, mais direta do que em linguagens imperativas tradicionais. Além disso, a forma como o código é limpo e claro também me surpreendeu bastante, pois, a leitura é fluida e compreensível, e o reconhecimento das expressões e instruções de lógica torna-se muito simples após os primeiros contatos.

De um ponto de vista geral, a linguagem Haskell realmente me surpreendeu de forma

que não achei que iria, pois no início adquiri certo preconceito pela linguagem, pelo fato de possuir um paradigma muito diferente do que estou acostumada, mas depois deste estudo, e compreendendo o básico da linguagem, me senti bem confortável, e na verdade, percebi, que ela não é tão difícil quanto achei que seria, é bem mais fácil na verdade, mesmo que ainda não esteja acostumada. Gostei bastante desta pesquisa, e de poder conhecer esse paradigma diferente e totalmente novo para mim, além da linguagem em si, ela me permitiu ter uma visão mais ampla sobre linguagens de programação, mostrando caminhos mais curtos e lógicos para se chegar a um resultado, e o que eu considerava “difícil”, era somente por não conhecer, e porque teria que sair da zona de conforto das linguagens imperativas e compreender uma linguagem nova em paradigma funcional.

9.4 Guilherme Zuim

A experiência de pesquisar e aplicar a linguagem Haskell foi, para mim, um exercício de desconstrução. A transição do pensamento imperativo de linguagens como C e Java para o paradigma funcional puro gera um estranhamento inicial. No entanto, o que mais me impressionou não foi apenas a sintaxe concisa, mas a fundamentação matemática rigorosa que sustenta a linguagem, especialmente a sua herança direta do Cálculo Lambda. Além disso, a Avaliação Preguiçosa (lazy evaluation) foi um conceito revelador. Perceber que a linguagem consegue lidar com estruturas potencialmente infinitas, calculando apenas o necessário, demonstra uma sofisticação na gestão de recursos que eu ainda não tinha explorado. Essa característica permite escrever códigos muito mais modulares e elegantes, separando a lógica de geração de dados da lógica de consumo. Em conclusão, o estudo do Haskell foi uma oportunidade valiosa para expandir meus horizontes acadêmicos. Sair da zona de conforto das linguagens tradicionais me permitiu entender que existem caminhos mais diretos e logicamente rigorosos para a computação. Acredito que o aprendizado deste paradigma funcional não apenas me ensinou uma nova linguagem, mas me deu ferramentas mentais para escrever códigos mais limpos e eficientes em qualquer outra tecnologia.

Referências

- [1] Paul Adrien Maurice Dirac. *The Principles of Quantum Mechanics*. International series of monographs on physics. Clarendon Press, 1981.
- [2] Albert Einstein. Zur Elektrodynamik bewegter Körper. (German) [On the electrodynamics of moving bodies]. *Annalen der Physik*, 322(10):891–921, 1905.
- [3] Paul Hudak, John Hughes, Simon Peyton Jones, and Philip Wadler. A history of haskell: Being lazy with class. *Proceedings of the Third ACM SIGPLAN Conference on History of Programming Languages*, June 2007.
- [4] Donald Knuth. Knuth: Computers and typesetting.
- [5] Miran Lipovača. *Learn You a Haskell for Great Good!: A Beginner’s Guide*. No Starch Press, San Francisco, 2011.